LAPORAN PRAKTIKUM PBO 6

POLYMORPHISM



Disusun Oleh:

Ananda Firdaus (251511003)

Jurusan Teknik Komputer dan Informatika

Rabu, 7 Oktober 2026

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2025

Repo: https://github.com/FirdausSinau/PBO

BAGIAN A - Another Type of Employee

A.1 Hierarchy

```
            StaffMember          <<abstract>>
             /          \
      Volunteer        Employee
                       /      \
                Executive      Hourly
                                  \
                               Commission     <-- ditambahkan
```

A.2 Array Polimorfik di Staff.java

```
public class Staff
{
    StaffMember[] staffList;

    public Staff()
    {
        staffList = new StaffMember[8];
        ...
    }

    public void payday()
    {
        double amount;

        for (int count = 0; count < staffList.length; count++)
        {
            System.out.println(staffList[count]);

            amount = staffList[count].pay();  // polymorphic

            if (amount == 0.0)
                System.out.println("Mksh y!");
            else
                System.out.println("Paid: " + amount);

            System.out.println("-----------------------------------");
        }
    }
}
```

Tipe array adalah `StaffMember[]`. Array ini dapat menampung `Executive`, `Employee`, `Hourly`, `Volunteer`, dan `Commission` karena kelimanya adalah `StaffMember`.

A.3 Code Commission.java

```
public class Commission extends Hourly
{
    private double totalSales;
    private double commissionRate;
```

```
    public Commission(String eName, String eAddress, String ePhone,
                      String socSecNumber, double rate, double commissionRate)
    {
        super(eName, eAddress, ePhone, socSecNumber, rate);

        this.commissionRate = commissionRate;
        totalSales = 0;
    }

    public void addSales(double sales)
    {
        totalSales += sales;
    }

    @Override
    public double pay()
    {
        double commission = totalSales * commissionRate;

        totalSales = 0;

        return super.pay() + commission;
    }

    @Override
    public String toString()
    {
        String result = super.toString();

        result += "\nTotal Sales: " + totalSales;

        return result;
    }
}
```

A.4 Dua Commission pada staffList

```
Commission c6 = new Commission("Rina", "Jl. Merdeka 10",
        "555-1122", "111-22-3333", 6.25, 0.20);
c6.addHours(35);
c6.addSales(400);
staffList[6] = c6;

Commission c7 = new Commission("Bayu", "Jl. Sudirman 25",
        "555-3344", "444-55-6666", 9.75, 0.15);
c7.addHours(40);
c7.addSales(950);
staffList[7] = c7;
```

Reference Commission c6 sebagai persiapan/setup, karena addHours() dan addSales() tidak terlihat melalui reference StaffMember.

A.5 Output payday()

Name: Sam

Address: 123 Main Line

Phone: 555-0469

Social Security Number: 123-45-6789

Paid: 2923.07

Name: Carla

Address: 456 Off Line

Phone: 555-0101

Social Security Number: 987-65-4321

Paid: 1246.15

Name: Woody

Address: 789 Off Rocker

Phone: 555-0000

Social Security Number: 010-20-3040

Paid: 1169.23

Name: Diane

Address: 678 Fifth Ave.

Phone: 555-0690

Social Security Number: 958-47-3625

Current hours: 40

Paid: 422.0

Name: Norm

Address: 987 Suds Blvd.

Phone: 555-8374

Mksh y!

Name: Cliff

Address: 321 Duds Lane

Phone: 555-7282

Mksh y!

Name: Rina

Address: Jl. Merdeka 10

Phone: 555-1122

Social Security Number: 111-22-3333

Current hours: 35

Total Sales: 400.0

Paid: 298.75

Name: Bayu

Address: Jl. Sudirman 25

Phone: 555-3344

Social Security Number: 444-55-6666

Current hours: 40

Total Sales: 950.0

Paid: 532.5

A.6 Perbandingan: dengan if/else vs polimorfisme

Sebagai pembanding, loop yang sama jika ditulis **tanpa** polimorfisme:

```
if (s instanceof Executive)      amount = ((Executive) s).pay();
else if (s instanceof Commission) amount = ((Commission) s).pay();
else if (s instanceof Hourly)     amount = ((Hourly) s).pay();
else if (s instanceof Employee)   amount = ((Employee) s).pay();
else if (s instanceof Volunteer)  amount = ((Volunteer) s).pay();
```

Perbandingan:

Variabel	Versi polimorfisme	Versi if/else
Baris kode	1	5 cabang, terus bertambah
Tambah tipe baru	**0 baris berubah**	+1 cabang `else if`
Lupa menambah cabang	tidak mungkin	tipe itu tidak terproses

BAGIAN B - Painting Shapes

B.1 Hierarchy

```
                Shape                <<abstract>>
            - shapeName (private)
            + area()                 <<abstract>>
            + toString()

          /          |          \
    Rectangle     Sphere     Cylinder
```

B.2 Code area() Tiap Subclass

Shape.java

```
public abstract class Shape
{
    private String shapeName;

    public Shape(String name)
    {
        shapeName = name;
    }

    public abstract double area();

    @Override
    public String toString()
    {
        return shapeName;
    }
}
```

Rectangle.java

```
public class Rectangle extends Shape
{
    private double length;
    private double width;

    public Rectangle(double length, double width)
    {
        super("Rectangle");
        this.length = length;
        this.width = width;
    }

    @Override
    public double area()
    {
        return length * width;
    }

    @Override
    public String toString()
    {
        return super.toString() + " of length " + length + " and width " + width;
    }
}
```

Sphere.java

```
public class Sphere extends Shape
{
    private double radius;

    public Sphere(double radius)
    {
        super("Sphere");
        this.radius = radius;
    }

    @Override
    public double area()
    {
        return 4 * Math.PI * radius * radius;
    }

    @Override
    public String toString()
    {
        return super.toString() + " of radius " + radius;
    }
}
```

Cylinder.java

```
public class Cylinder extends Shape
{
    private double radius;
    private double height;

    public Cylinder(double radius, double height)
    {
        super("Cylinder");
        this.radius = radius;
        this.height = height;
    }

    @Override
    public double area()
    {
        return Math.PI * radius * radius * height;
    }

    @Override
    public String toString()
    {
        return super.toString() + " of radius " + radius + " and height " + height;
    }
}
```

B.3 Code Paint.amount(Shape s)

```
public class Paint
{
    private double coverage;  // number of square feet per gallon

    public Paint(double c)
    {
        coverage = c;
    }

    public double amount(Shape s)
```

```
    {
        System.out.println("Computing amount for " + s);
        return s.area() / coverage;
    }
}
```

B.4 Output Kebutuhan Cat

```
Computing amount for Rectangle of length 20.0 and width 35.0
Computing amount for Sphere of radius 15.0
Computing amount for Cylinder of radius 10.0 and height 30.0


Number of gallons of paint needed...
Deck 2
Big Ball 8.1
Tank 26.9
```

`amount()` yang sama menghitung tiga bentuk dengan rumus area yang berbeda.

Perhitungan (coverage = 350):

Object	area()	amount()
Rectangle 20 x 35	700.0	2
Sphere r=15	2827.43	8.078
Cylinder r=10 h=30	9424.78	26.928

B.5 Uji Polymorphic Reference (Step 11)

Ditambahkan pada `PaintThings`:

```
Shape s;

s = deck;
System.out.println(s + " area=" + s.area());

s = bigBall;
System.out.println(s + " area=" + s.area());

s = tank;
System.out.println(s + " area=" + s.area());
```

Output:

```
--- Polymorphic reference ---
Rectangle of length 20.0 and width 35.0 area=700.0
Sphere of radius 15.0 area=2827.4333882308138
Cylinder of radius 10.0 and height 30.0 area=9424.77796076938
```

Terbukti, menggunakan miliknya masing-masing.

B.6 Bukti Abstract Class Tidak Dapat Diinstansiasi

Eksperimen: `Shape s = new Shape("X");`

```
error: Shape is abstract; cannot be instantiated
        Shape s = new Shape("X");
                  ^
1 error
```

`Shape` hanya mewajibkan `area()`. Object `Shape` akan punya method kosong.

B.7 Bukti Abstract Method Wajib Diimplementasikan

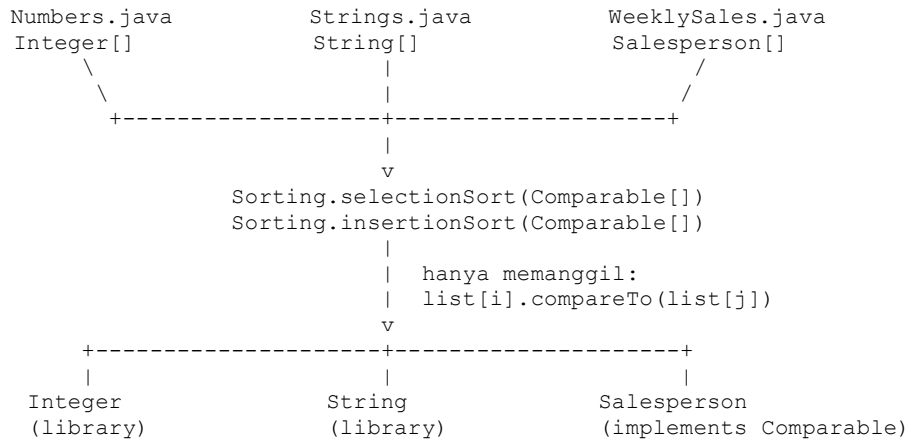
Eksperimen: menghapus `area()` dari `Cylinder`.

```
error: Cylinder is not abstract and does not override abstract method area() in Shape
public class Cylinder extends Shape
       ^
1 error
```

Beda dengan `toString()`, kalau `Sphere` lupa menulis `toString()`, compile tetap bisa tapi yang dipakai adalah versi `Shape` (mengembalikan `"Sphere"` saja).

BAGIAN C - Polymorphic Sorting

C.1 Struktur dan Alur



Tiga program terpisah, masing-masing punya `main()` sendiri. Yang sama hanya satu: class `Sorting`.

C.2 Error Awal int[] (T6-06)

`Numbers.java` versi awal memakai `int[]`:

```
int[] intList;
...
intList = new int[size];
...
Sorting.selectionSort(intList);
```

Error compiler:

```
error: incompatible types: int[] cannot be converted to Comparable[]
        Sorting.selectionSort(intList);
                              ^
1 error
```

`Sorting` menerima `Comparable[]`. `int` primitive bukan object, tidak punya method, tidak bisa memenuhi requirement `Comparable`. Errornya bukan di sintaks, tapi nunjukin jika algoritma sorting bergantung pada `compareTo()`.

C.3 Perbaikan ke Integer[]

Dua baris yang diubah:

```
Integer[] intList;
...
intList = new Integer[size];
```

Baris `intList[i] = scan.nextInt();` tidak diubah, otomatis mengubah `int` menjadi `Integer`.

Output (input: 5 angka - 3 1 4 1 5):

```
How many integers do you want to sort?
Enter the numbers...

Your numbers in sorted order...
5 4 3 1 1
```

C.4 Strings.java - Algoritma yang Sama

```
String[] stringList;
...
stringList = new String[size];
...
Sorting.insertionSort(stringList);
```

Output (input: pepaya, apel, mangga, jeruk):

```
How many strings do you want to sort?
Enter the strings...


Your strings in sorted order...
pepaya  mangga  jeruk  apel
```

`Sorting` sama, tanpa satu baris pun diubah, mengurutkan `String[]`.

C.5 Perubahan insertionSort ke Descending

Perubahan hanya pada satu operator:

```
// ASCENDING (asli)
while (position > 0 && key.compareTo(list[position - 1]) < 0)

// DESCENDING (diubah)
while (position > 0 && key.compareTo(list[position - 1]) > 0)
```

Kondisi dibaca "selama key lebih besar dari elemen di kirinya, geser ke kiri". Karena yang berubah adalah algoritmanya, arah pengurutan terbalik untuk smeua tipe yang memakai `insertionSort`.

`Numbers.java` dan `Strings.java` kemudian diubah memanggil `insertionSort` (bukan `selectionSort`), dan hasilnya menjadi descending:

```
Numbers : 5  4  3  1  1
Strings : pepaya  mangga  jeruk  apel
```

C.6 Salesperson.compareTo()

```
public class Salesperson implements Comparable
{
    private String firstName, lastName;
    private int totalSales;
    ...
    public int compareTo(Object other)
    {
        Salesperson otherSales = (Salesperson) other;

        int result;

        // (1) UTAMA: bandingkan total sales
        if (getSales() < otherSales.getSales())
            result = -1;                        // aku lebih kecil
        else if (getSales() > otherSales.getSales())
            result = 1;                         // aku lebih besar
        else
        {
            // (2) SALES SAMA -> tie-breaker: nama belakang
            result = getLastName().compareTo(otherSales.getLastName());

            // (3) NAMA BELAKANG SAMA JUGA -> bandingkan nama depan
            if (result == 0)
                result = getFirstName().compareTo(otherSales.getFirstName());
```

```
            }

            return result;
        }
        ...
    }
```

Bagian	Penjelasan
(Salesperson) other	Casting diperlukan karena parameter bertipe Object. Karena pakai Comparable mentah
if / else if untuk sales	Mengikuti konvensi Comparable: negatif = lebih kecil, positif = lebih besar.
getLastName().compareTo(...)	Memanggil String.compareTo() milik library. Jadi compareTo buatan sendiri disusun dari compareTo library

C.7 Output WeeklySales

```
Ranking of Sales for the Week


Taylor, Harry:   7300
Adams, Andy:     5000
Duck, Daffy:     4935
Smith, Walt:     3000
Jones, Jane:     3000
Jones, James:    3000
Black, Jane:     3000
Doe, Jim:        2850
Walter, Dick:    2800
Trump, Don:      1570
```

Urutan mengikuti total sales dari terbesar ke terkecil. Empat orang dengan sales sama (3000) diurutkan berdasarkan nama secara reverse alphabetical.

C.8 Bukti Interface Benar-benar Kontrak

Eksperimen: menghapus `compareTo()` dari `Salesperson` sementara `implements Comparable` tetap dipertahankan.

```
error: Salesperson is not abstract and does not override abstract method compareTo(Object)
in Comparable
public class Salesperson implements Comparable
       ^
1 error
```

Compiler nuntut class memenuhi seluruh method yang dideklarasikan di interface.

TABEL SKENARIO UJI T6-01 s.d. T6-09

ID	Skenario	Expected Result	Bukti	Status
T6-01	pay() dipanggil melalui StaffMember reference	Implementation mengikuti actual subtype	Rina (Commission) menghasilkan 298.75	**PASS**
T6-02	Commission menerima hours dan sales	pay = hourly pay + commission	Rina: 218.75 + 80.00 = 298.75; Bayu: 390.00 + 142.50 = 532.50	**PASS**
T6-03	payday() menangani subtype baru	Loop umum tidak perlu cek concrete class	staffList dari 6 menjadi 8 object, payday() tidak diubah sama sekali	**PASS**
T6-04	Paint.amount(deck/bigBall/tank)	Method yang sama memanggil area() subtype yang sesuai	Deck 2, Big Ball 8.1, Tank 26.9 dari satu method amount(Shape)	**PASS**
T6-05	Shape reference diarahkan ke tiga object	area() berubah sesuai actual object	Satu variabel s: area=700.0 lalu 2827.43 lalu 9424.78	**PASS**
T6-06	int[] diberikan ke Sorting	???	error: incompatible types: int[] cannot be converted to Comparable[]	**PASS**
T6-07	Integer[] dan String[] di-sort	Keduanya dapat memakai Sorting yang sama	5 4 3 1 1 dan pepaya mangga jeruk apel, satu Sorting, tidak ad algoritma baru	**PASS**
T6-08	Salesperson.compareTo dipakai Sorting	Urutan mengikuti sales, baru tie-breaker nama	Empat orang sales 3000 terurut reverse alphabetical: Smith, Jones (Jane), Jones (James), Black	**PASS**
T6-09	insertionSort descending	Output Numbers, Strings, WeeklySales descending	5 4 3 1 1 / pepaya mangga jeruk apel / 7300 ... 1570	**PASS**

5. Definition of Done

#	Pernyataan	Status
1	Baseline Staff berhasil dijalankan dan dapat menunjukkan satu polymorphic call pada payday()	Done - staffList[count].pay(), Staff.java:66
2	Commission berhasil ditambahkan sebagai subtype Hourly dan pay() menghasilkan kombinasi hourly pay + commission	Done - 298.75 dan 532.50

3	Loop payroll tetap bekerja tanpa if/switch berdasarkan concrete class	Done - payday() tidak diubah
4	Paint.amount(Shape) berhasil digunakan untuk Rectangle, Sphere, dan Cylinder	Done - 2 / 8.1 / 26.9
5	Dapat menunjukkan satu variabel Shape yang merujuk ke beberapa subtype	Done - Step 11, area 700.0 / 2827.43 / 9424.78
6	Menyimpan bukti error int[] ketika diberikan ke Sorting	Done - C.2
7	Integer[] dan String[] berhasil diurutkan dengan class Sorting yang sama	Done - C.3, C.4
8	Salesperson memenuhi Comparable dan dapat diurutkan melalui compareTo()	Done - C.6, C.7
9	Dapat menjelaskan perbedaan reference type dan actual object type dari tiga kasus	Refleksi 1, 2, 7

6. Self Assessment

Pernyataan	Skor 1-5
Saya dapat membedakan reference type dan actual object type.	3
Saya memahami mengapa overridden method dipilih saat runtime.	3
Saya dapat menggunakan superclass/abstract class sebagai polymorphic reference.	3
Saya dapat menjelaskan polymorphism melalui interface Comparable.	3
Saya dapat menambah subtype baru tanpa mengubah code umum yang hanya bergantung pada abstraction.	3

7. Refleksi

1. Pada loop payday(), mengapa satu pemanggilan staffList[count].pay() dapat menghasilkan perhitungan yang berbeda?

Karena object yang dirujuk setiap elemen berbeda tipe. staffList[0] berisi Executive, staffList[3] berisi Hourly, staffList[6] berisi Commission. Walaupun array-nya bertipe StaffMember, Java menentukan implementasi pay() berdasarkan tipe object aseli nya, bukan tipe array-nya.

2. Jika reference bertipe StaffMember menunjuk ke object Commission, apa yang ditentukan oleh reference type dan apa yang ditentukan oleh actual object?

Reference type menentukan apa yang boleh dipanggil. Compiler membaca StaffMember, jadi pay() boleh sedangkan .addSales() tidak boleh.

3. Mengapa Paint.amount(Shape s) tidak memerlukan if/else untuk membedakan Sphere, Rectangle, dan Cylinder?

Karena s.area() sudah diselesaikan otomatis oleh Java berdasarkan tipe object. amount() tidak perlu tau bedanya Sphere, Rectangle, atau Cylinder.

Kalau memakai if/else, setiap bentuk baru berarti satu cabang baru di amount(). Nambahin manual, dan kalua kelewatan jadi tidak terhitung.

4. Apa hubungan abstract method area() dengan dynamic binding pada kasus Painting Shapes?

Karena area() abstract di Shape, compiler memaksa setiap subclass punya. Jadi setiap object Shape pasti punya area().

Kalau area() tidak abstract dan subclass boleh melewatkannya, Java tidak punya jaminan dan pemanggilan lewat reference Shape bisa nabrak method yang tidak ada.

5. Mengapa Sorting dapat mengurutkan Integer, String, dan Salesperson tanpa mengetahui struktur internal masing-masing class?

Karena Sorting berinteraksi lewat: compareTo().

Dia tidak pernah mengakses totalSales, value, atau isi karakter String. Yang dia butuhkan cuma hasil perbandingan.

Integer menghitung selisih angka. String menangani encoding Latin1 dan UTF16. Salesperson membandingkan sales lalu nama.

6. Mengapa int[] gagal digunakan pada Sorting tetapi Integer[] berhasil?

Karena int adalah primitive, bukan object. Primitive tidak punya method sama sekali.

Integer adalah object yang membungkus int, dan dia sudah implements Comparable.

Sorting menerima Comparable[], dan hanya object yang bisa memenuhinya. Errornya pun bukan soal sintaks, tapi soal tipe: int[] cannot be converted to Comparable[].

7. Apa perbedaan polymorphism via inheritance pada StaffMember/Shape dengan polymorphism via interface pada Comparable?

Perbandingan	via inheritance	via interface
Bentuk	abstract class	interface
Kata kunci	extends	implements
Membawa data	ya (name, shapeName)	tidak
Boleh punya method konkrit	ya (toString())	tidak
Boleh berapa	1	banyak
Kapan dipakai	kalau ada data yang perlu diwariskan	kalau cuma butuh kemampuan

Di Case 1 dan Case 2, subclass butuh data dari induknya.

Di Case 3, Integer, String, dan Salesperson tidak berbagi data, punya induk masing-masing (Number atau Object). Jadi extends lagi tidak mungkin, maka dipakai lah interface.

8. Lengkapi: "Polymorphism bukan berarti object berubah menjadi object lain, tetapi ..."

"...tetapi satu pemanggilan yang sama dapat menghasilkan perilaku berbeda" + karena Java memilih implementasi method berdasarkan tipe object yang sebenarnya - pada saat program berjalan.